

DATA STRUCTURE AND ALGORITHM-1

- **DATA STRUCTURE:** A Data Structure is a specialized format for organizing and storing data so that operations on the can be performed efficiently.

Think of it like organizing items in real life.

- **Bookshelf** → Books are arranged on shelves, making them easy to find.
- **Dictionary** → Words are organized alphabetically for quick lookup.
- **Filing cabinet** → Documents are stored in labeled folders.

Similarly, a Data Structure organizes data so a program can perform operations like searching, inserting, deleting, and update efficiently.

There are 2 Types of DS,

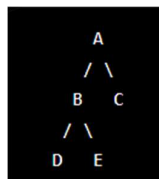
- **LINEAR DS:** Elements are arranged sequentially.

Eg: Array, LinkedList, Stack, Queue

10 → 20 → 30 → 40

- **NON-LINEAR DS:** Elements are organized hierarchically or as networks.

Eg: Tree, Graph, Heap, Tire



❖ Built-in vs Custom DS

- **Built-in DS (PYTHON)** → List, Tuple, Dictionary, Set
- **Custom DS** → These are implemented manually or provided by libraries.
Eg: LinkedList, Stack, Queue, Graph, Binary Tree

- **ALGORITHMS:** An Algorithm is a finite sequence of well-defined instructions that take input, processes it, and produces the desired output to solve a problem.

Think of it as MAKING A TEA,

Boil Water→Add Tea Leaves→Add Sugar→Boil for some time→Serve.

Every Algorithms have 3 components: Input, Processing, Output.

Analogy

Imagine a library:

- **Data Structure** → The way books are arranged on shelves.
- **Algorithm** → The process you follow to find a specific book.

Both work together: good organization makes the search process faster.

- **PROGRAMMING LANGUAGE EXECUTION:** This explain how your source code is translated in to machine instructions.

- **COMPILER**→ It translates the entire program in to machine code before execution.

Eg: c, c++, GO, Rust

- **INTERPRETER**→ It executes the program while it runs.

Eg: Python, JavaScript, Ruby

NOTE: Python is not purely interpreted, because python first **compiles** to **bytecode (.pyc)** then the python virtual machine interprets the bytecode.

- **TYPING:** It answers when does the language know the type of variables.

- **STATIC TYPING:** The variable type is known before executing (generally at compile time).

```
</> C
int age = 20;
```

- **DYNAMIC TYPING:** The variables type is determined while the program runs.

```
</> Python
x = 10
```

MEMORY ALLOCATION: It is the process of reserving memory In RAM for variables, objects, or data structures during the program execution.

- RAM = Hotel
- Memory allocation = Assigning a room to a guest
- Variable = Guest
- Memory deallocation = Guest checks out, room becomes available

2 Types of memory allocations,

- **Static memory allocation:** Memory is allocated before the program starts (Compile time)
- **Dynamic memory allocation:** Memory is allocated while the program is running (at runtime)

NOTE: Python doesn't use static memory allocation because it doesn't know what object will exist until runtime.

NOTE: C can use both static and dynamic memory allocation, because static typing and static memory allocation are different.

<pre><> C int x;</pre>	<pre><> C int *arr = malloc(100*sizeof(int));</pre>
Statically typed ✓	Statically typed ✓
Static memory ✓	Dynamic memory ✓

- **MEMORY LEAK:** It occurs when a program keeps allocating memory allocated even though it has no longer needs it, preventing that memory from being reused.

NOTE: In python, garbage collection automatically reclaims unused memory

- Garbage Collection: GC is the automatic process of finding and freeing memory occupied by objects that are no longer being used by program.

For 99% of Python programs, you don't need to use the garbage collector directly. Python manages memory automatically.

- **COMPLEXITY ANALYSIS:** Complexity Analysis is the process of measuring how an algorithms performance changes as input size increases.

We mainly measure 2 things,

- Time Complexity: How long does an algorithm takes to run as the input size increases.
- Space Complexity: How much an extra memory an algorithm use as the input size grows.

Time complexity measures how the number of operations performed by an algorithm grows as the input size(n) increases. **ASYMPTOTIC NOTATION** is a mathematical notation used to describe the growth rate of an algorithms time and space complexity as the input size become very large.

- **ASYMPTOTIC NOTATION:** It is a mathematical notation that is used to describe the growth rate of an algorithms time and space complexity as the input size increases

Types of Asymptotic Notation,

- BIG O Notation (O) → Upper Bound (Worst Case): It describes the maximum amount of work an algorithm can perform as the input size grows. (Most Of the companies use this notation)

NOTE: BIG O tells us the worst-case performance of an algorithm.

- BIG OMEGA (Ω) → Lower Bound (Best Case): It describes the minimum amount of work an algorithm performs.

NOTE: BIG OMEGA tells us the best-case performance.

- BIG THETA (Θ) → Tight Bound (Average/Exact Growth): It describes the exact growth rate when the upper and lower bound are the same.

NOTE: BIG THETA tells us the typical growth of an algorithm when its best and worst case grow at the same rate.

- **TYPES OF TIME COMPLEXITY:** It is classified on the basis on how the number of operations grows as the input size (n) increases.

Common Types of Time Complexity		
Time Complexity	Growth	Example
$O(1)$	Constant	Accessing an array element
$O(\log n)$	Logarithmic	Binary Search
$O(n)$	Linear	Linear Search
$O(n \log n)$	Linearithmic	Merge Sort
$O(n^2)$	Quadratic	Bubble Sort
$O(n^3)$	Cubic	Triple nested loops
$O(2^n)$	Exponential	Recursive Fibonacci
$O(n!)$	Factorial	Generating all permutations

- Constant time – $O(1)$: The number of operations does not depend on the input size.
- Linear Time – $O(n)$: The number of operations grow directly with n .
- Quadratic Time – $O(n^2)$: Usually caused by 2 nested loops.
- Cubic Time – $O(n^3)$: Usually caused by 3 nested loops.
- Logarithmic Time – $O(\log n)$: Here, the input reduced by half in each step.
- Linearithmic Time – $O(n \log n)$: Occurs in efficient sorting algorithms like merge sort.
- Exponential Time – $O(2^n)$: The number of operations doubles with each increase in input.
- Factorial Time – $O(n!)$: The slowest common complexity, occurs when generating all possible arrangements (permutations).

- **SPACE COMPLEXITY:** It refers to the amount of memory an algorithm uses as a function of input size. It's a measure of how much memory an algorithm requires to execute and solve a problem.

- **RECURSION:** The process in which a function call itself directly or indirectly is called recursion and the corresponding function is called recursive function.

NOTE: Every Recursion have 2 cases,

- A) Base Case: This tells the function to stop calling itself.
- B) Recursive Case: This is the part where function call itself.

```
def print_numbers(n):
    #Base Case
    if n==0:
        return
    #Recursive Case
    print_numbers(n-1)
    print(n,end=' ')
print_numbers(5)
```

Q) Why recursion when loop exists?

Ans: Recursion is used because it can solve problems that are naturally divided into smaller version of the same problems, making the solution simpler than using loops.

Recursion is classified into 7 main types,

- **Direct Recursion** → A function call itself directly.

```
<> Python
def fun(n):
    if n == 0:
        return

    print(n)
    fun(n - 1)

fun(5)
```

- Indirect (Mutual) Recursion → One functions call another function, and eventually that function calls the first function.

```
<> Python
def even(n):
    if n == 0:
        print("Even")
        return
    odd(n - 1)

def odd(n):
    if n == 0:
        print("Odd")
        return
    even(n - 1)

even(4)
```

- **Tail Recursion** → The recursion call is the last statement in the function.

```

Python
def print_reverse(n):
    if n == 0:
        return

    print(n)

    print_reverse(n - 1)

```

➤ **Head Recursion** → The recursive call happens before any other work.

```

Python
def print_forward(n):
    if n == 0:
        return

    print_forward(n - 1)

    print(n)

```

➤ **Tree Recursion** → Each function call creates more than one recursive call

```

Python
def fib(n):
    if n <= 1:
        return n

    return fib(n-1) + fib(n-2)

```

➤ **Linear Recursion** → Each function creates only one recursive call.

```

Python
def count(n):
    if n == 0:
        return

    count(n-1)

```

➤ **Nested Recursion** → A recursive call is used inside another recursive call.

```

Python
def fun(n):
    if n > 100:
        return n - 10

    return fun(fun(n + 11))

```

NOTE: Equation uses in DSA

$$\text{LCM}(a, b) = \frac{a \times b}{\text{gcd}(a, b)}$$

- ARRAY: An ARRAY is a linear DS that stores multiple elements of same data type in a contiguous memory location.

NOTE: Python does not have a built-in ARRAY. Instead, it uses LIST.

FOR DSA we treat python LIST as ARRAYS.

CHARACTERISTICS OF ARRAY:

- ✓ Stores multiple values
- ✓ Elements are arranged sequentially
- ✓ Access using an index
- ✓ Fast random Access
- ✓ Usually stores the same type of data (Conceptually)

ADVANTAGES OF ARRAY:

- ✓ Fast access
- ✓ Easy traversal
- ✓ Less memory overhead
- ✓ Cache friendly

DISADVANTAGES OF ARRAY:

- ✓ Slow insertion at beginning or middle
- ✓ Slow deletion

- STATIC VS DYNAMIC ARRAY:

- STATIC ARRAY: Its size is fixed
Eg: C, you cannot increase it later
- DYNAMIC ARRAY: Size changes automatically
Eg: Python LIST

- ❖ PYTHON LIST: A LIST is a built-in python data structure to store multiple elements in a single variable. It is ORDERED, MUTABLE, and allows DUPLICATE values.

OPERATIONS ON LIST:

- Append () → Adds an element at the end.
- Insert () → Insert at any position.
- Pop () → Remove last element: Remove specific index.
- Remove () → Remove a specific element.
- Clear () → Delete all elements.
- Extend () → Add another list.
- Copy () → Creates copy.
- Reverse () → To reverse.

- Sort ()→ To sort in ascending order (reverse=True will give in descending.)
 - Count ()→ Count occurrence.
 - Index ()→ Returns the index of first occurrence.
 - Sorted ()→ Returns a new sorted list.
- ❖ SHALLOW COPY AND DEEP COPY: A shallow copy creates a new outer object, but the nested objects are still shared between the original and the copies object. A DEEP COPY creates a completely independent copy of an object, including all nested objects.
- ❖ PYTHON TUPLE: A TUPLE is a built-in python DS uses to store multiple elements in a single variable. It is ordered, immutable (Cannot change after creation), and allows duplicate values.

TUPLE OPERATIONS:

- CONCATENATION (+)
 - REPETATION (*)
 - MEMBERSHIP
- ❖ PYTHON SETS: A set is collection of UNIQUE data, meaning that elements within a set cannot be duplicated. It is unordered and INDEXING is not possible in set operation.

SET OPERATION:

- UNION |
 - INTERSECTON &
 - DIFFERENCE –
 - SYMMTRIC_DIFFERENCE ^
- ❖ PYTHON DICTIONARIES: It is a collection of items, similar to lists and tuples. However, unlike lists and tuples, each item in a dictionary is a key-value pair (Consisting of a key and a value pair).
- NOTE: You can't have same key for 2 elements.**
- ❖ HASHING IN PYTHON: Hashing is a technique used in DS that efficiently stores and retrieves data in a way that allows for quick access.
- ❖ SEARCHING ALGORITHM: These are essential tools in computer science used to locate specific item within a collection of data.

Two types of searching:

- LINEAR SEARCH: Is searching algorithm that checks each element sequentially until the target element is found or the array ends.
- BINARY SEARCH: Is searching algorithm that repeatedly divides a sorted array into two halves by comparing the target with the middle element, reducing the search space by half each time.

NOTE: Binary search is better than linear search because it reduces the search space by half after every comparison [Time Complexity: $O(\log 2 n)$ making it much faster for searching in large sorted dataset. Linear search checks one element in at a time, [Time complexity: $O(n)$].

LOWER AND UPPER BOUND IN BS:

- LB: It Is the index of the first element that is greater than or equal to the target
- UB: It is the index of the first element that is strictly greater than the target.

NOTE: LB and UB are variation of BS used when we need more than just checking whether an element exist. They help efficiently find the insertion position, first occurrence, count of occurrence, floor, cell, and next greater element all in $O(\log n)$ time.

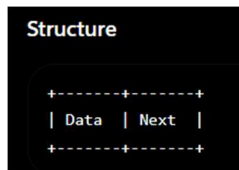
- ❖ LINKED LIST IN PYTHON: A Linked List is a DS made up of nodes, where each nodes stores data and a reference (pointer) to the next node. Unlike arrays, Linked List elements are not stored in contiguous memory.

OPERATIONS IN A LINKED LIST

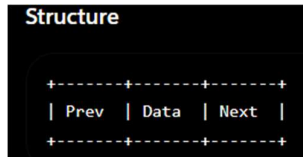
- TRAVERSE:
- SEARCH:
- INSERT:
- DELETE:
- UPDATE:
- REVERSE:
- COUNT NODES:

TYPES OF LINKED LISTS:

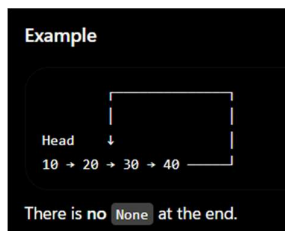
- A) SINGLY LIKED LIST (SSL): This is the most basic and most commonly used linked list. Each node has data and pointer to the next node.



- B) DOUBLY LINKED LIST (DLL): Each node has previous pointer, data and pointer to the next node.



- C) CIRCULAR SINGLY LINKED LIST (CSLL): Same as SLL, but the last node points back to the first node.



- D) CIRCULAR DOUBLY LINKED LIST (CDLL): Each node has previous pointer and next pointer and the first and last node are connected.

Slow and Fast Pointer concept: The slow and fast pointer techniques use two pointers moving at different speed. The slow pointer moves one node at a time, while the fast pointer moves two nodes at a time. It is commonly used to find the middle of linked list, detect cycles, and solve other linked list problems in $O(n)$ time with $O(1)$ extra space.